

SafetyDial Phase 0 Report

Does the Safety-Gymnasium locomotion suite support the proposal’s premise?

Sachin Mohan
Student No. 2699183
Supervised by Geraud Nangue Tasse
University of the Witwatersrand

20 September 2026

Abstract

Phase 0 was built to test, cheaply and early, whether the environment chosen in the revised proposal can support its central claim. The answer is robot-dependent, and the proposal picked the wrong robot. Walker2d, the environment the revised proposal builds its argument around, has no irreversible failures at all: given a four second horizon and a competent planner it recovers from lying on the ground in every case tested, and recoverability does not fall below 0.88 at any point either side of the benchmark’s failure flag. Hopper appears at first to have a genuine irreversibility boundary sitting roughly 40 steps *after* the flag fires, but a sensitivity check dissolves it: relaxing the recovery predicate’s pitch tolerance from 5.7 to 8.6 degrees, still strictly inside the benchmark’s own healthy band, takes its irrecoverable fraction to zero. What that predicate was measuring is whether a hopping robot can hold near-static balance, not whether it can recover. Neither robot, then, carries the irreversible failure the proposal needs. This report records what was built, what was measured, the corrections that measurement forced, and what follows.

1 Verdict

The revised proposal rests on a contrast: that the velocity cost is *extrinsic and recoverable* while falling is *intrinsic and irreversible*, and that this split, inside a single recognised benchmark, gives a labelled mode and an unlabelled mode for the transfer experiment.

The second half of that contrast is false for Walker2d, which is the robot the proposal names. Measured on settled-fallen states, with the robot’s torso on the floor:

Robot	irrecoverable at $H = 250$ (2s)	at $H = 500$ (4s)
Walker2d	0.000	0.000
Hopper	0.333	0.167

Table 1: Fraction of fallen states from which no action sequence was found. 30 states per cell, full CEM search on the true simulator. Walker2d is at zero for both horizons.

Hopper looks at first like the exception. Sweeping either side of the failure flag with 40 states per point (Figure 5), Walker2d never drops below 0.88 recoverable anywhere, while Hopper falls from 1.000 at the flag to 0.450 at +40 steps, which reads as a real irreversibility boundary displaced from where **terminated** puts it.

It does not survive scrutiny. Section 4.4 re-labels the same states under weaker recovery predicates: relaxing the pitch tolerance from 0.10 to 0.15 rad, still well inside the benchmark’s own 0.2 rad healthy band, takes the irrecoverable fraction to zero, as does shortening the dwell requirement. Hopper’s apparent irreversibility is a statement about static balance, not about recovery.

What follows. Neither robot carries the irreversible failure the proposal’s experiment needs, so the locomotion suite should be replaced rather than re-parameterised. This is the outcome Phase 0 exists to detect, and detecting it cost one day rather than surfacing in Stage 2 in November.

2 What was built

2,964 lines across seven helper modules and five scripts, all lint-clean, with no `src/` package (per the project’s own rule that new code lives in `experiments/helpers/`).

Module	Role
<code>locoEnv.py</code>	Vendored Safety-Gymnasium velocity task; thresholds; stack fingerprint
<code>locoCollect.py</code>	State-only HDF5 writer and rollout loop
<code>locoPolicies.py</code>	Uniform <code>act(obs)</code> adapters
<code>locoData.py</code>	Lazy-render dataset; worker-safe EGL; render fingerprint
<code>locoMetrics.py</code>	Gate 0 statistics, disjointness lift, matched pairs, AUROC
<code>oracle.py</code>	The recoverability oracle. Five search tiers. Evaluation only
<code>equivCheck.py</code>	Vendored-env equivalence gates; numpy-only, loads under both stacks

2.1 Storage: state, not pixels

Restoring (`qpos`, `qvel`) and replaying the recorded actions reproduces `qpos` to 7.4×10^{-15} , and renders from a restored state are bit-identical, so pixels need never be written to disk. Walker2d has `na == 0`, making (`qpos`, `qvel`) the complete state.

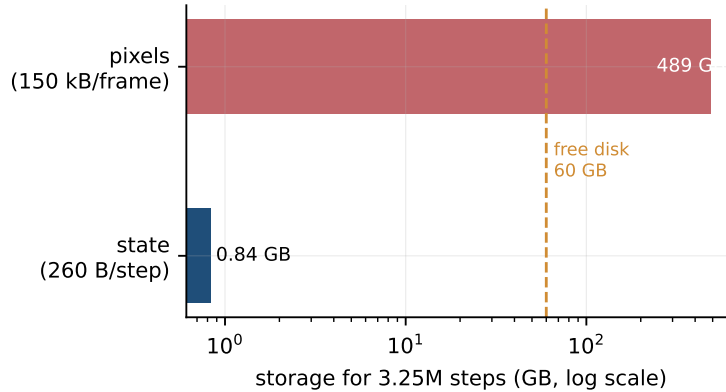


Figure 1: Storing state costs 260 bytes per step against 150 kB for a 224×224 frame. The pixel route does not fit in the available disk; the state route is free. Frames regenerate at 3,101 per second, and the dataset delivers 607 samples/s single-process, four to seven times what the GPU consumes.

2.2 The vendored environment

`safety-gymnasium==1.0.0` cannot be installed alongside `stable-worldmodel`: it carries hard == pins on `gymnasium 0.28` and `mujoco 2.3`, so `uv lock` fails outright. The velocity task is therefore copied verbatim rather than depended on. `walker2d.xml` has md5 `7fb5491cd0097896df56653c3d0f18a0` on both stacks, with identical model scalars, and an equivalence harness gates cost sums, termination indices, observations, velocities and rewards.

3 The recoverability oracle

The environment’s own `terminated` flag cannot serve as ground truth for irreversibility. It is a threshold on torso height and pitch, and both decode from a raw 32×32 grayscale frame at $R^2 = 0.99$, so scoring a detector against it is close to circular.

Recoverability is therefore measured directly. From a saved MuJoCo state, with termination disabled, the oracle searches for an action sequence that returns the robot to a settled upright pose. Any success proves recoverability, so the search is one-sided: more search can only shrink the irrecoverable set, never grow it.

Recovery predicate (Walker2d). $1.00 \leq z \leq 1.80$, $|\text{pitch}| \leq 0.50$, $\max |\dot{q}_{0:3}| \leq 5.0$, held for 25 consecutive steps. Every constant sits strictly inside the environment’s own health set ($0.8 < z < 2.0$, $|\text{pitch}| < 1.0$), which is what stops the oracle from restating `terminated`.

Five tiers, short-circuiting on first success. **T0** replays the actions that actually followed the state, a certificate rather than a search; **T π** a trained controller; **T1** constant torques; **T2** low-pass random torques; **T3** CEM over ten control knots interpolated to the horizon, on the true simulator. The knot parameterisation is essential: the raw space is 1500-dimensional at $H = 250$ and CEM does not work there, while ten knots makes it 60-dimensional.

Validation V1 (benign upright states must come back recoverable) reaches 1.000 on 40 states, against 0.92 for the original design. The T0 certificate makes it 59 times cheaper, 4 ms per state against 235 ms.

4 Findings

4.1 The benchmark declares failure long before the robot falls

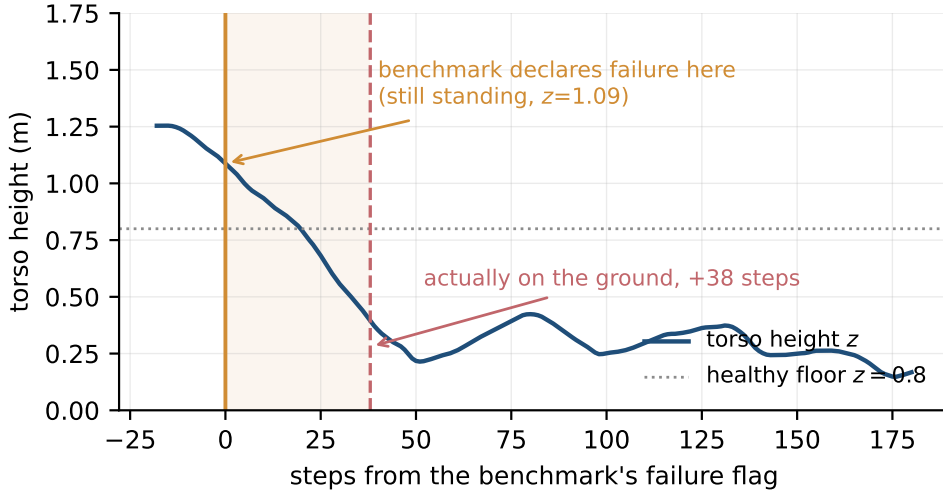


Figure 2: One episode, collected with termination disabled so it continues past the benchmark’s cutoff. The flag fires while the torso is still at $z \approx 1.09$, leaning past 55 degrees. The robot does not reach the ground for roughly another 38 steps.

4.2 Falling is not irreversible

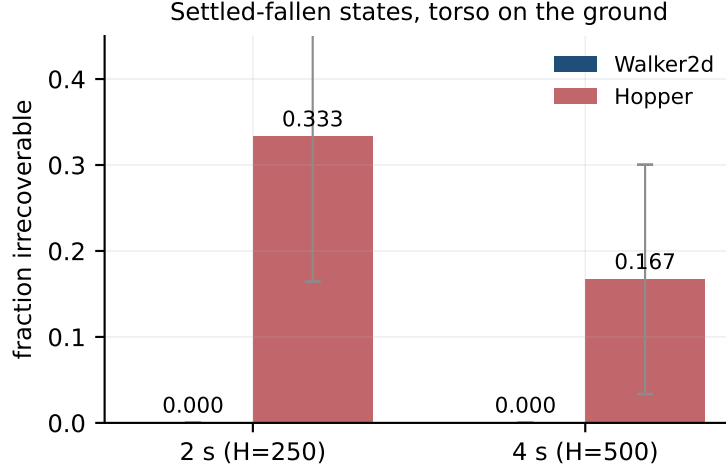


Figure 3: Irrecoverable fraction on settled-fallen states, by robot and search horizon. Error bars are normal approximations at $n = 30$. Walker2d is at zero for both horizons; Hopper declines from 0.333 to 0.167 as the horizon doubles, so what is being measured is the time recovery takes, not whether it is possible.

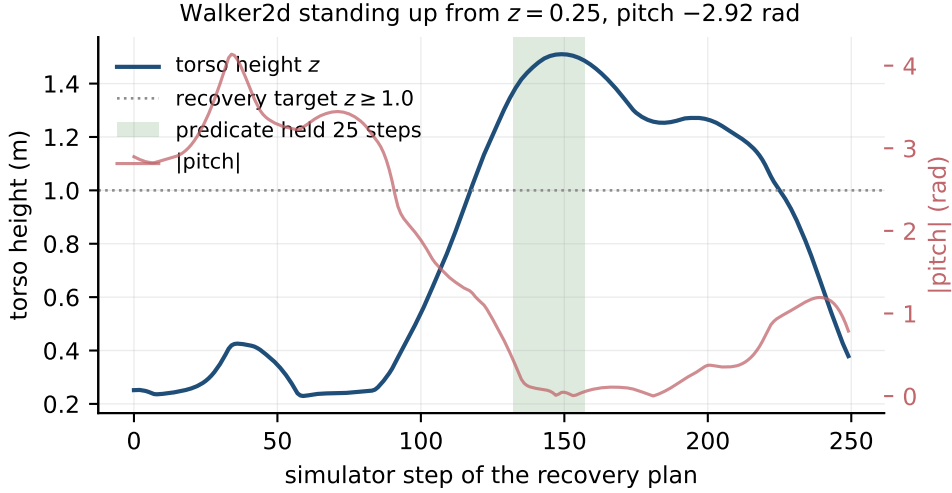


Figure 4: A replayed recovery. The robot starts lying on the ground, upside down, and every action in the plan is inside $[-1, 1]$, the same range any policy has. It rolls, pushes off and stands. This is real physics rather than a simulator exploit: MuJoCo’s Walker2d is a planar model with gear-100 actuators and no self-collision, so it can right itself.

4.3 Recoverability either side of the failure flag

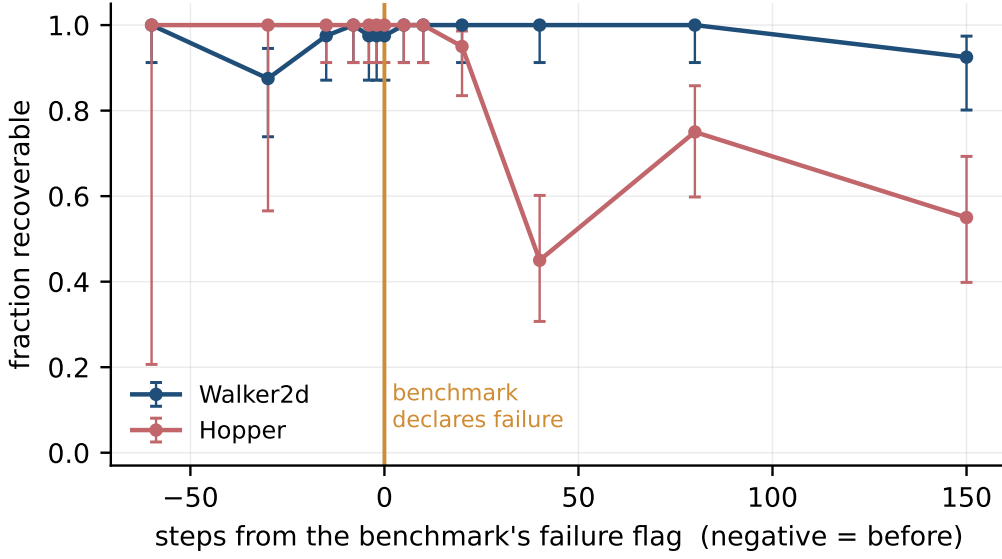


Figure 5: Fraction of states from which a recovery was found, against offset from the benchmark’s failure flag. Error bars are Wilson 95% intervals at $n = 40$ per point. If the flag marked a transition into irreversibility, these curves would fall sharply to zero. Walker2d never moves: recoverability stays near one on both sides, including 150 steps after the flag with the robot on the ground. Hopper does fall, but late, from 1.000 at the flag to 0.450 at +40 steps, so its irreversibility boundary is real and is displaced roughly 40 steps past where the benchmark places it. The Hopper curve is not monotone beyond +40; at $n = 40$ those points are within each other’s intervals and the sampled configurations differ.

4.4 Hopper’s boundary is a property of the predicate, not the dynamics

The obvious question about Figure 5 is whether Hopper’s drop is real or an artefact of how “recovered” was defined. Hopper’s predicate asks for $|\text{pitch}| \leq 0.10$ rad and root velocity ≤ 5.0 held for 25 consecutive steps, and a hopper is an inherently dynamic system, so it may be being asked to stand still rather than to recover.

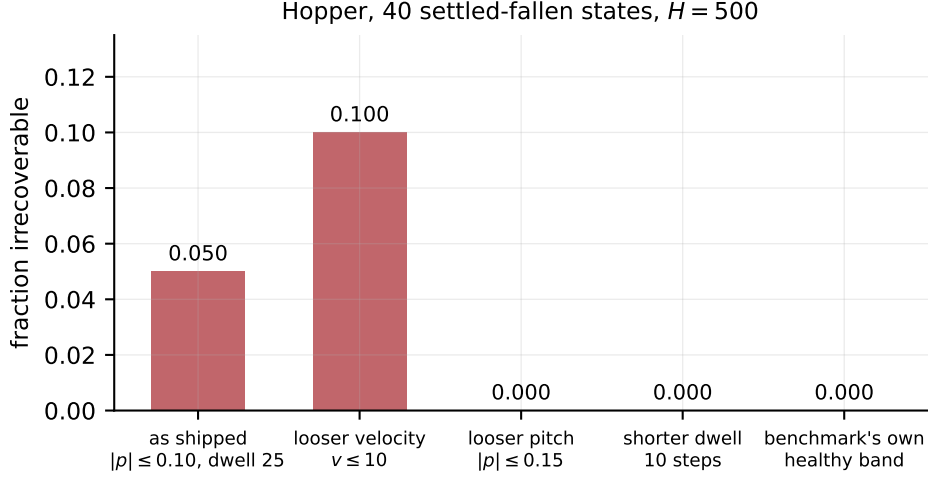


Figure 6: The same 40 settled-fallen Hopper states, re-labelled under weaker recovery predicates at $H = 500$. Relaxing pitch from 0.10 to 0.15 rad, or shortening the dwell from 25 steps to 10, takes the irrecoverable fraction to zero. Every variant shown is still strictly inside the benchmark’s own healthy band of $|\text{pitch}| < 0.2$.

The horizon behaves the same way: at $H = 250$ roughly 45% of states at +150 are irrecoverable, and at $H = 500$ that falls to 5%. Both knobs point the same way, and the conclusion is the one already drawn for Walker2d: what is being measured is how long recovery takes and how tidily it must finish, not whether recovery is possible.

This is the same class of error as the joint-velocity cap corrected in Section 5, caught in one place and left standing in another. It is recorded here rather than quietly fixed because it changes the report’s recommendation.

4.5 Mode A and Mode B are not disjoint

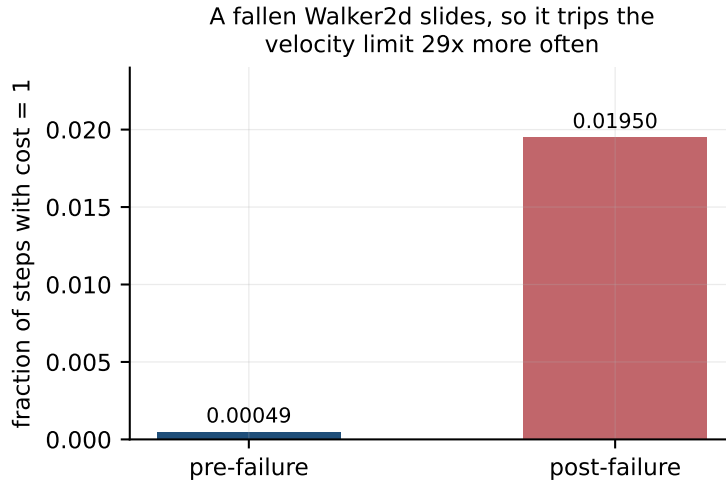


Figure 7: Cost rate before and after the failure. This asymmetry is what inverted the Gate 0 statistic: control windows were being drawn from the post-failure region, so the denominator was measuring cost caused *by* the fall.

This corrects a number reported earlier in the work. An initial run gave a disjointness lift of 0.288, that is, cost appearing *anti-correlated* with imminent failure, which would have been

ideal for the transfer experiment. It was a bug. Corrected, with control windows confined to the pre-failure phase and length-matched to the numerator:

Quantity	Value
cost rate, pre-failure phase	0.00049
$P(\text{cost in the 125 steps before failure})$	0.035 (7 of 200 episodes)
$P(\text{cost in a matched control window})$	0.000 to 0.001
disjointness lift	35 to ∞ across control seeds

Table 2: Gate 0 required lift ≤ 2.0 . Both rates are very small, so the ratio is unstable and the honest statement is that the gate fails, not that the lift takes any particular value.

Read carefully: cost is *rare* before a failure in absolute terms, but when it does occur it is concentrated immediately before one. Note also that this is measured under a crude scripted-forward policy that drives maximum torque, accelerates past the limit and then trips; a velocity-constrained policy would plausibly behave differently.

5 Corrections forced by measurement

1. **The velocity cap applies to the root degrees of freedom only.** Walker2d joint velocities routinely exceed 10 rad/s in normal gait, so an all-DOF cap turned “recovered” into “recovered and then stood perfectly still”. Measured: 91% of benign states satisfy an all-DOF cap against 98.9% for a root-only cap, and that 91% was exactly the 92% figure an earlier pilot had reported as its validation ceiling.
2. **A certificate tier was added.** Pure search systematically under-detects recoverability for *dynamic* states, because a robot in mid-gait needs an actual balancing controller to stay upright and zero torque simply drops it.
3. **Collection runs with termination disabled.** The rollout records state *before* each action, so `terminated[t]` means “action t made it unhealthy” while `qpos[t]` is the last *healthy* state, and the episode then ends. Every row in such a dataset is healthy: it contains no failures at all. This produced a false alarm in which states at `terminated == 1` came back 96% recoverable and read as the premise collapsing; they were healthy standing robots.

6 Adversarial audit

A five-dimension audit with three independent skeptics per finding: 134 agents, 43 findings raised, and roughly two thirds refuted under verification. Defects confirmed and fixed:

Severity	Defect
critical	<code>disjointness_lift</code> drew control windows from the post-failure region, inverting Gate 0 from fail to pass
high	The recovery predicate had <i>no upper z bound</i> , so a ballistic arc could satisfy it: 25 steps is only 0.2s of flight
high	Oracle labels were not reproducible. <code>set_state</code> leaves <code>qacc_warmstart</code> dirty, so the same rollout returned -1.4618128503645678 or $\dots 664$ depending on what ran before
high	The G4 equivalence gate was inverted: <code>first_term</code> is -1 when no termination occurs, so “reference terminated at step 0, test never terminated” computed as an off-by-one and was excused as floating-point noise
high	The equivalence harness emitted observations, velocities, rewards and per-step costs, shipped them across, then compared only totals
high	T2’s low-pass filter had no gain compensation: at $\beta = 0.9$ it commanded rms torque 0.132 out of 1.0, making the search four times weaker than intended
medium	Truncated rollouts returned <code>margins.max()</code> , an optimistic and different quantity from the windowed minimum they claimed to be
medium	CEM rollouts were excluded from the rollout count, so the budget-saturation check read a near-constant

Two items are recorded rather than fixed: the lead-time statistic is a minimum over a sparse offset grid and is censored, so it understates; and the hopeless early exit is the one place the oracle’s one-sidedness is an assumption rather than a guarantee.

7 Where this leaves the project

The oracle and the pipeline are sound and environment-agnostic. The environment is not. Four options:

1. **Controller-relative viability.** Keep Safety-Gymnasium and define irrecoverable as “outside the viability kernel of a realistic controller class” rather than “no omniscient planner can recover”. Viability kernels in control theory are always defined relative to admissible controls, and a safety filter protects a *specific* policy, so this is arguably the right object. The $T\pi$ tier already implements it. The cost is that the failure set stops being purely a property of the dynamics.
2. **OGBench-Cube.** The August proposal’s environment. A cube pushed off the table is genuinely irrecoverable. The cost is the loss of published safe-RL baselines.
3. **Report the negative result.** “Standard safe-RL locomotion benchmarks have no irreversible failures, and their termination flags are hand-set constants a competent controller recovers from.” Surprising, checkable, and largely already done. The cost is that it becomes a benchmark critique rather than the Safety Dial thesis.
4. **Add genuine irreversibility to the environment.** The cost is that it is no longer the literal published benchmark, which undercuts the reason for choosing it.

An earlier draft of this report recommended option 1 applied to Hopper, on the strength of Figure 5 alone. Section 4.4 withdraws that recommendation: once the predicate is relaxed anywhere, Hopper’s irrecoverable set empties, so keeping the locomotion suite would mean defending a threshold rather than measuring a property.

That leaves options 2 and 3 as the live ones, and they compose: the oracle and the negative result are worth reporting whichever environment comes next, and an environment with a genuinely one-way transition, such as a cube pushed off a table, is what the rest of the programme needs. Option 1 remains available but only with a controller class fixed in advance and justified on its own terms, not chosen after seeing which threshold produces failures.

Reproducing

```
# storage and replay invariants
uv run python experiments/scripts/verify_replay.py
# the full Phase 0 triage
uv run python experiments/scripts/run_triage.py
# every figure in this report
uv run python experiments/scripts/make_phase0_figures.py
```

The numbers behind every figure are committed beside it in `docs/phase0/data/`, so a plot can be re-made or checked without re-running the simulation that produced it. Supporting records: `notes/phase0Report.md` (the same material in prose), `notes/safetyGymLocomotion.md` (measured environment facts) and `notes/terminationIsNotIrreversibility.md` (the finding in detail).